

Week 6 Applied Class Worksheet (Solutions)

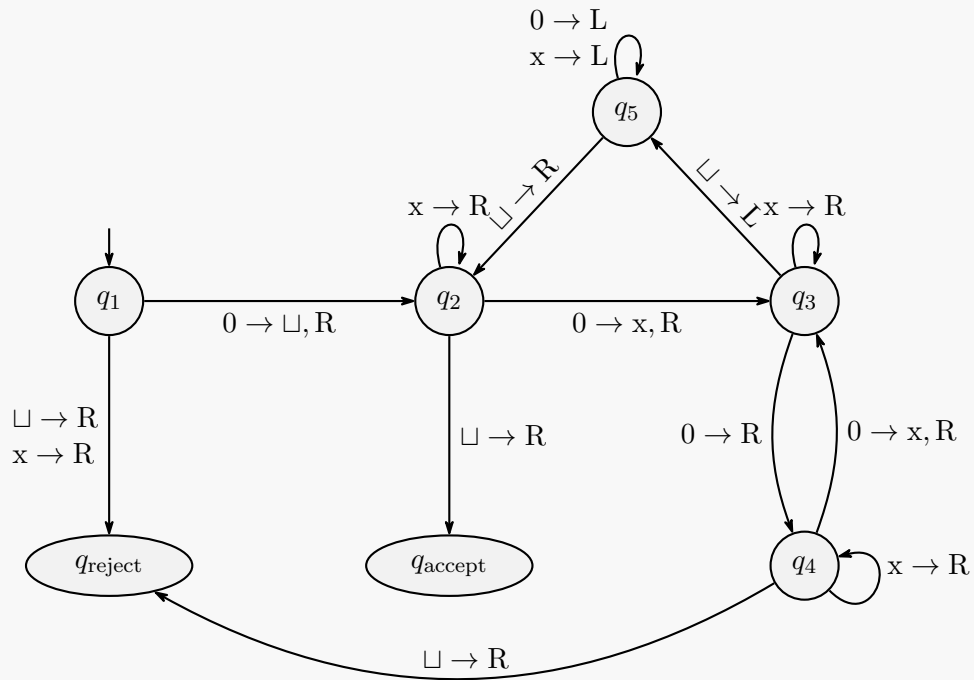
COMP2048

Dr. Kasra Khosoussi

Semester 1, 2026

Task 1. *Example 3.7 from Sipser*

Consider the following Turing machine that decides $L = \{0^{2^n} \mid n \geq 0\}$.



In this Turing machine, $\Sigma = \{0\}$ and $\Gamma = \{0, x, \sqcup\}$. Some transitions in the diagram are labeled using the shorthand notation (e.g., $0 \rightarrow R$) for cases those transitions where the head reads and writes the same symbol (or, equivalently, doesn't alter that tape cell). For example, $0 \rightarrow R$ means the head upon reading a 0 just moves to the right.

- (a) Give the sequence of configurations that this machine enters when started on each of the following input strings. Use the notation we saw in the lecture for configurations.
- ε
 0
 00
 000
 0000
- (b) Determine which inputs are accepted or rejected, and verify your answer assuming this machine is deciding L .
- (c) Explain the algorithm that this machine is implementing for deciding L .

Solution.

We write configurations by inserting the current state immediately before the tape symbol being scanned.

- (a) Configuration sequences: See handwritten pages at the end.
- (b) Accepted/rejected inputs.

Input	Outcome
ε	reject
0	accept
00	accept
000	reject
0000	accept

This matches the language

$$L = \{0^{2^n} \mid n \geq 0\} = \{0, 00, 0000, 00000000, \dots\}.$$

Indeed, among the given strings, the accepted ones are exactly those of lengths

$$1 = 2^0, \quad 2 = 2^1, \quad 4 = 2^2.$$

- (c) Algorithm.

The machine checks whether the number of 0's is a power of 2.

It works as follows (from Sipser, Example 3.7):

M_2 = "On input string w :

- (i) Sweep left to right across the tape, crossing off every other 0.
- (ii) If in stage 1 the tape contained a single 0, accept.
- (iii) If in stage 1 the tape contained more than a single 0 and the number of 0s was odd, reject.

- (iv) Return the head to the left-hand end of the tape.
- (v) Go to stage 1.”

At the beginning, the first 0 is replaced with a blank to mark the leftmost cell of the tape for our future right-to-left sweeps. Each iteration of stage 1 cuts the number of 0s in half. As the machine sweeps across the tape in stage 1, it keeps track of whether the number of 0s seen is even or odd. If that number is odd and greater than 1, the original number of 0s in the input could not have been a power of 2. Therefore, the machine rejects in this instance. However, if the number of 0s seen is 1, the original number must have been a power of 2. So in this case, the machine accepts.

So the machine is repeatedly halving the number of surviving 0's. It accepts exactly when this repeated halving ends at 1, which happens exactly when the original length is a power of 2.

Task 2. Turing Machine Design

Design a Turing machine for deciding $L = \{0^n 1^{2^n} \mid n \geq 0\}$. Explain the algorithm behind your design.

Hint: Adapt the machine we saw in the lecture for deciding $\{0^n 1^n \mid n \geq 0\}$.

Solution.

See handwritten pages at the end. The algorithm is very similar to what we had in the class. Instead of marking one 0 from the beginning, and one 1 from the end (in this case marking is done by writing blank), we mark one 0 and mark two 1s from the end. We could have introduced another symbol for marking 0s and 1s.

Task 3. Turing-recognizable vs. Turing-decidable

Explain why every Turing-decidable language is Turing-recognizable.

Solution.

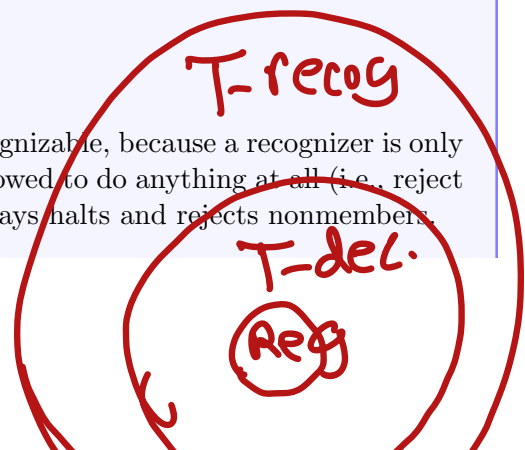
A language is *Turing-decidable* if there exists a Turing machine that halts on *every* input and accepts exactly the strings in the language.

A language is *Turing-recognizable* if there exists a Turing machine that accepts exactly the strings in the language; on strings not in the language, it may either reject or loop forever.

Now suppose L is Turing-decidable. This means that there is a machine M such that:

- if $w \in L$, then M accepts w ;
- if $w \notin L$, then M rejects w .

But this already satisfies the definition of Turing-recognizable, because a recognizer is only required to accept members of the language, and it is allowed to do anything at all (i.e., reject or looping) on nonmembers. M does even better: it always halts and rejects nonmembers.



Therefore every Turing-decidable language is Turing-recognizable.

Task 4. *Turing Machines and Regular Languages*

Let $M = (Q, \Sigma, \delta, q_0, F)$ be a DFA.

- (a) Construct a Turing machine that decides $L(M)$ (i.e., accepts an input string w if $w \in L(M)$, and rejects it otherwise). Use the formal notation (i.e., 7-tuple) in your construction).
- (b) Explain why this construction proves that the set of regular languages is a subset of the set of Turing-decidable languages.

Solution.

- (a) Construction.

Let the DFA be

$$M = (Q, \Sigma, \delta, q_0, F).$$

We construct a Turing machine

$$T = (Q', \Sigma, \Gamma, \delta', q_0, q_{\text{acc}}, q_{\text{rej}})$$

that simulates M on the input from left to right.

Define:

$$Q' = Q \cup \{q_{\text{acc}}, q_{\text{rej}}\}, \quad \Gamma = \Sigma \cup \{\sqcup\}.$$

Intuitively, whenever the TM is in state $q \in Q$ and scans a symbol $a \in \Sigma$, it behaves like the DFA transition to $\delta(q, a)$ and moves right.

Formally:

- For each $q \in Q$ and $a \in \Sigma$,

$$\delta'(q, a) = (\delta(q, a), a, R).$$

- When the TM reaches the first blank after the input, it decides according to whether the current simulated DFA state is accepting:

$$\delta'(q, \sqcup) = \begin{cases} (q_{\text{acc}}, \sqcup, R), & \text{if } q \in F, \\ (q_{\text{rej}}, \sqcup, R), & \text{if } q \notin F. \end{cases}$$

This machine never changes the tape contents; it just reads the input once from left to right and keeps the simulated DFA state in its finite control.

Why it works. After reading any prefix x of the input, the TM is in exactly the DFA state that M would be in after processing x . Therefore, when it reaches the blank after the whole input, it accepts exactly when the DFA would accept.

- (b) A language is regular iff some DFA recognizes it. Given any DFA for a regular language, the construction above produces a Turing machine that halts on every input and accepts exactly the same language.

So for every regular language L , there exists a Turing machine that decides L . Hence

$$\{\text{regular languages}\} \subseteq \{\text{Turing-decidable languages}\}.$$

In other words, every regular language is Turing-decidable.

$\epsilon:$
 $q_1 \sqcup \sqcup \dots$ X
 $\sqcup q_{\text{reject}} \sqcup \sqcup \dots$

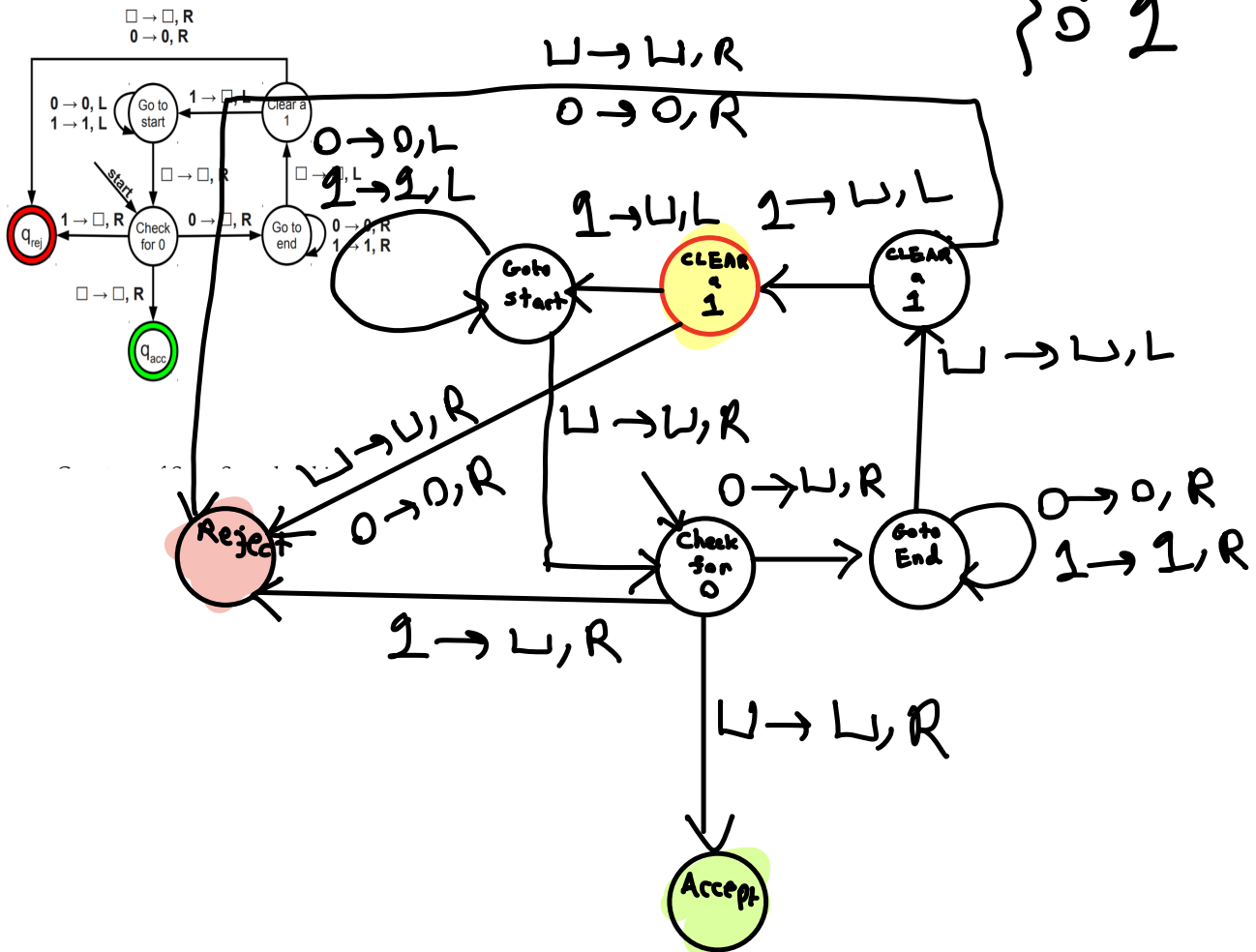
$0:$
 $q_1 0 \sqcup \dots$
 $\sqcup q_2 \sqcup \dots$
 $\sqcup \sqcup q \sqcup \dots$
 accept ✓

$00:$
 $q_1 00 \sqcup \dots$
 $\sqcup q_2 0 \sqcup \dots$
 $\sqcup x q_3 \sqcup \dots$
 $\sqcup q_5 x \sqcup \dots$
 $q_5 \sqcup x \sqcup \dots$
 $\sqcup q_2 x \sqcup \dots$
 $\sqcup x q_2 \sqcup \dots$
 $\sqcup x \sqcup q \sqcup \dots$
 accept ✓

$000:$
 $q_1 000 \sqcup$
 $\sqcup q_2 00 \sqcup \dots$
 $\sqcup x q_3 0 \sqcup \dots$
 $\sqcup x 0 q_4 \sqcup \dots$
 $\sqcup x 0 \sqcup q_{\text{reject}} \sqcup \dots$ X

$0000:$ $q_1 0000 \sqcup \dots$ $\sqcup q_2 000 \sqcup \dots$ $\sqcup x q_3 00 \sqcup \dots$ $\sqcup x 0 q_4 0 \sqcup \dots$ $\sqcup x 0 x q_5 \sqcup \dots$ $\sqcup x 0 q_5 x \sqcup \dots$ $\sqcup x q_5 0 x \sqcup \dots$ $\sqcup q_5 x 0 x \sqcup \dots$ $q_5 \sqcup x 0 x \sqcup \dots$ $\sqcup q_2 x 0 x \sqcup \dots$ $\sqcup x q_2 0 x \sqcup \dots$ $\sqcup x x q_3 x \sqcup \dots$ $\sqcup x x x q_3 \sqcup \dots$ $\sqcup x x q_5 x \sqcup \dots$	$\sqcup x q_5 x x \sqcup \dots$ $\sqcup q_5 x x x \sqcup \dots$ $q_5 \sqcup x x x \sqcup \dots$ $\sqcup q_2 x x x \sqcup \dots$ $\sqcup x q_2 x x \sqcup \dots$ $\sqcup x x q_2 x \sqcup \dots$ $\sqcup x x x q_2 \sqcup \dots$ $\sqcup x x x \sqcup q \sqcup \dots$ accept ✓
--	---

$$\{0^n 1^{2^n} \mid n \geq 0\}$$



$$\{0^n 1^n \mid n \geq 0\}$$